



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善



第三章 处理机调度与死锁

第1讲



本章内容

3.1 处理机调度层次与调度算法目标

3.2 作业调度

3.3 进程调度

3.5 死锁概述

3.6 预防死锁

3.7 避免死锁





3.1 处理机调度层次与调度算法目标

□ 3.1.1 处理机调度的层次

□ 3.1.2 调度算法的目标



3.1.1 处理机调度的层次

- 在多道程序系统中，调度的**实质**是一种**资源分配**。
- 处理机调度：根据一定的**算法和原则**对处理机资源进行**重新分配**的过程。
- 前提：作业/进程数远大于处理机数。
- 目的：提高资源利用率，减少处理机空闲时间。
- 调度算法应满足：特定用户需求（快速响应）；系统整体效率（平均周转时间短）；调度算法的开销。



3.1.1 处理机调度的层次

1. 高级调度（**High Level Scheduling**）
2. 低级调度（**Low Level Scheduling**）
3. 中级调度（**Intermediate Scheduling**）



1. 高级调度（High Level Scheduling）

- 高级调度又称**作业调度**或长调度，其调度对象是**作业**。用于决定把外存上后备队列中哪些作业调入内存，并为它们创建进程、分配必要的资源，然后将新建进程插入到**就绪队列**中，准备运行。
只调入一次，调出一次
- 高级调度主要应用于**多道批处理系统**中，而在分时和实时系统中不设置。

每次作业调度，都需做以下两个决定：

- 接纳**多少**个作业——取决于**多道程序度**
- 接纳**哪些**作业——取决于**调度算法**

▲ 数目太多，会影响系统的服务质量。如，周转时间长。

▲ 数目太少，会导致系统资源利用率和系统吞吐量低。



2. 低级调度 (Low Level Scheduling)

- 低级调度又称**进程调度**或短调度，其调度对象是“**进程**”或“**内核级线程**”。用于决定就绪队列中的哪个进程应获得处理机，然后再由分派程序执行把处理机分配给该进程的具体操作。
- 低级调度是**最基本**的一种调度，**频率非常高**。在**多道批处理**、**分时**和**实时**三种类型的OS中，都必须配置这一级调度。

调度方式：

- 非抢占式
- 抢占式



3. 中级调度 (Intermediate Scheduling)

□ 中级调度又称内存调度。其目的是为了提高内存的利用率和系统的吞吐量。引入虚拟存储技术后，把那些暂时不能运行的进程，调至外存（即对换区）等待，此时进程的状态为就绪驻外存状态（或挂起状态）。当具备运行条件且内存有空闲时，从挂起队列中选择合适进程再重新调入内存，并修改其状态为就绪状态。

挂起和激活，存储器管理中的对换功能。



3. 中级调度 (Intermediate Scheduling)

□ 三种调度相比较:

• 进程调度——运行频率最高

• 作业调度——频率最低

• 中级调度——介于两者之间

- 基于上两者之间
- 中程调度

- 10-100ms进行一次调度
- 为了避免调度本身占用太多CPU时间，不宜使进程调度算法太复杂
- 短程调度

- 一般发生在一批作业运行完毕并退出系统，又需要重新调入一批作业进入内存时
- 几分钟一次
- 长程调度

1. 进程从创建态转换到就绪态的工作由 [填空1] 调度完成；进程由就绪态转换到执行态的工作由 [填空2] 调度完成；中级调度又称 [填空3] 调度，其目的是提高 [填空4] 的利用率和系统吞吐量。

作答





3.1.2 调度算法的目标

1. 处理机调度算法的共同目标
2. 批处理系统的目标
3. 分时系统的目标
4. 实时系统的目标



1. 处理机调度算法的共同目标

(1) 资源利用率

为提高系统的资源利用率，应使系统中的处理机和其它所有资源都尽可能地**保持忙碌状态**。

处理机的利用率可用以下方法计算：

$$\text{CPU的利用率} = \frac{\text{CPU的有效工作时间}}{\text{CPU的有效工作时间} + \text{CPU空闲等待时间}}$$



例：在有一个CPU和两个输入/输出设备（磁盘机和磁带机）的多道程序系统中，同时投入运行2个程序A和B。这2个程序对CPU、磁盘和磁带的使用顺序和使用时间为：

程序A：磁带（30S）、CPU（10S）、磁盘（30S）、
CPU（10S）、磁带（20S）

程序B：磁盘（20S）、CPU（30S）、磁带（40S）

假定：CPU、磁盘和磁带都能并行工作

试问：在单道和多道两种方式下，

1. 程序A和B从投入运行到运行完成所用的时间分别是多少？
2. CPU、磁盘和磁带的利用率是多少？



1. 处理机调度算法的共同目标

(2) 公平性

公平性是指应使诸进程都获得合理的CPU时间，不会发生进程饥饿现象。

(3) 平衡性

系统中可能有多种类型的进程，有的属于计算型作业，有的属于I/O型。为使系统中的CPU和各种外部设备都能经常处于忙碌状态，调度算法应尽可能保持系统资源使用的平衡性。

(4) 策略强制执行

对所制订的策略其中包括安全策略，只要需要，就必须予以准确地执行，即使会造成某些工作的延迟也要执行。



2. 批处理系统的目标

(1) 平均周转时间短

□ 周转时间：从作业提交给系统开始，到作业完成为止的时间间隔。

$$T_i = T_{e_i} - T_{s_i}$$

□ 包括四部分时间：

- 作业在外存后备队列上等待作业调度的时间
- 进程在就绪队列上等待进程调度的时间
- 进程在CPU上执行的时间
- 进程等待I/O操作完成的时间

平均周转时间

$$T = \frac{1}{n} \left[\sum_{i=1}^n T_i \right]$$

n是作业个数



2. 批处理系统的目标

(1) 平均周转时间短

□ 带权周转时间：作业的周转时间 T_i 与系统实际为它提供服务的时间 T_s 之比。即：

$$W = \frac{T_i}{T_s}$$

$$W \geq 1$$

作业的实际运行时间

平均带权周转时间

$$W = \frac{1}{n} \sum_{i=1}^n \frac{T_i}{T_s}$$



2. 批处理系统的目标

举例说明：

作业	进入时间	运行时间/min	开始时间	结束时间	周转时间/min	带权周转时间/min	平均周转时间/min	平均带权周转时间/min
JOB1	8:00	120	8:00	10:00	120	1	120	1.7
JOB2	8:50	50	10:00	10:50	120	2.4		



2. 批处理系统的目标

(2) 系统吞吐量高

吞吐量是指在单位时间内系统所完成的作业数，因而它与批处理作业的平均长度有关。

$$\text{系统吞吐量} = \frac{\text{完成作业总数}}{\text{总时间}}$$

若单纯为了获得高的系统吞吐量，应尽量多选择短作业运行。



2. 批处理系统的目标

(3) 处理机利用率高

处理机的利用率是衡量系统性能的十分重要的指标；而调度方式和算法又对处理机的利用率起着十分重要的作用。

若单纯为使处理机利用率高，应尽量多选择计算量大的作业运行。



3. 分时系统的目标

(1) 响应时间快

- **响应时间：**从用户通过键盘提交一个请求开始，直到屏幕上显示出处理结果为止的一段时间间隔。
- **包括三部分时间：**
 - 请求信息从键盘输入开始，直至将其传送到处理机的时间
 - 处理机对请求信息进行处理的时间
 - 将所形成的的响应信息回送到终端显示器的时间



3. 分时系统的目标

(2) 均衡性

- ❑ **均衡性：**系统响应时间的快慢应与用户所请求服务的复杂性相适应。
- ❑ 用户通常对较复杂任务的响应时间允许较长，对较简单任务的响应时间要短。



4. 实时系统的目标

(1) 截止时间的保证

- **截止时间**：某任务必须开始执行的最迟时间，或必须完成的最迟时间。
- **实时系统中**，调度算法的一个主要目标是保证实时任务对截止时间的要求。
 - **HRT任务（硬实时任务）**：调度方式和调度算法必须确保对截止时间的要求，否则可能造成难以预料的后果；
 - **SRT任务（软实时任务）**：调度方式和调度算法应基本上能保证对截止时间的要求。



3.2 作业调度

- 3.2.1 批处理系统中的作业
- 3.2.2 作业调度的主要任务
- 3.2.3 先来先服务和短作业优先调度算法
- 3.2.4 优先级调度算法和高响应比优先调度算法



3.2.1 批处理系统中的作业

- 作业：用户一次请求计算机系统为其完成任务所做工作的总和。它通常包括程序、数据和作业说明书，系统根据说明书对程序进行控制。
- 作业步：指每个作业在运行期间，都必须经过若干个相对独立，又相互关联的顺序加工步骤，其中的每一个加工步骤称为作业步。一个典型的作业可分成三个作业步：
 - ① “编译”作业步
 - ② “连结装配”作业步
 - ③ “运行”作业步



3.2.1 批处理系统中的作业

❑ 作业控制块（**Job Control Block, JCB**）：多道批处理系统为每个作业设置了一个作业控制块JCB，它是作业在系统中存在的标志，其中保存了系统对作业进行管理和调度所需的全部信息。

❑ 作业状态：

操作员把用户提交的作业通过某种输入方式输入到硬盘上，再为该作业建立JCB，并把它放入作业后备队列中。此时的作业状态称为“后备状态”。

- 后备状态

作业被作业调度选中后，便为它分配必要的资源和建立进程，并将它放入就绪队列。直到运行结束前，都处于“运行阶段”。

- 运行状态

- 完成状态

当作业完成或发生异常情况提前结束时，作业就进入“完成阶段”，此时作业的状态为“完成状态”。



3.2.2 作业调度的主要任务

□ 作业调度的主要任务：根据JCB中的信息，检查系统中的资源能否满足作业对资源的需求，以及按照一定的调度算法，从外存的后备队列中选取某些作业调入内存，并为它们创建进程、分配必要的资源。然后再将新创建的进程排在就绪队列上等待调度。

每次作业调度，都需做以下两个决定：

- 接纳**多少**个作业——取决于**多道程序度**
- 接纳**哪些**作业——取决于**调度算法**

▲ 数目太多，会影响系统的服务质量。如，周转时间长。

▲ 数目太少，会导致系统资源利用率和系统吞吐量低。



3.2.3 先来先服务和短作业优先调度算法

1. 先来先服务调度算法

2. 短作业优先调度算法

各种调度算法的学习思路6点：

算法思想；算法原理；适用于作业调度还是进程调度；调度方式；
优缺点；是否导致**饥饿**（即长期得不到服务）。



1. 先来先服务调度算法 非抢占式

□ 先来先服务（**first-come first-served, FCFS**）调度算法是一种最简单的调度算法。按照作业/进程到达的先后顺序进行调度。既可用于**作业调度**，也可用于**进程调度**。

思想：从公平角度考虑，等待时间越长优先级越高
类似排队打水

用于作业
调度中：

按照作业到达**后备队列**的先后次序来进行调度，或者说优先考虑在系统中等待时间长的作业。

用于进程
调度时：

从**就绪队列**中，选择一个最先进入该队列的进程，为之分配处理机，使之投入运行。该进程一直运行到完成或发生某事件而阻塞后，才放弃处理机。

优点：公平、算法简单易实现；不会导致饥饿。

1. 先来先服务调度算法

【例3-1】在单道系统中用FCFS算法调度如下作业，请完成下表。

作业名	A	B	C	D	E	平 均
到达时间	9:00	9:10	9:30	10:00	10:15	
服务时间	30分钟	1小时	10分钟	50分钟	20分钟	
完成时间	9:30	10:30	10:40	11:30	11:50	
周转时间	30分钟	80分钟	70分钟	90分钟	95分钟	73分钟
带权周转时间	1	1.33	7	1.8	4.75	3.176

调度顺序为：

A	B	C	D	E	
9:00	9:30	10:30	10:40	11:30	11:50

缺点：有利于长作业（进程），不利于短作业（进程）。有利于CPU繁忙型作业（进程），不利于I/O繁忙型作业（进程）。

1. 先来先服务调度算法

【例3-1】在单道系统中用FCFS算法调度如下作业，请完成下表。

作业名	A	B	C	D	E	平 均
到达时间	9:00	9:10	9:30	10:00	10:15	
服务时间	30分钟	1小时	10分钟	50分钟	20分钟	
完成时间	9:30	10:30	10:40	11:30	11:50	
周转时间	30分钟	80分钟	70分钟	90分钟	95分钟	73分钟
带权周转时间	1	1.33	7	1.8	4.75	3.176

- CPU繁忙型作业：需要大量的CPU时间进行计算，而很少请求I/O。如，科学计算；
- I/O繁忙型作业：是指CPU进行处理时，需频繁地请求I/O。如，大多数事务处理。



1. 先来先服务（FCFS）调度算法

练习：

作业名	进入时间	运行时间/min	开始时间	结束时间	周转时间/min	带权周转时间	平均周转时间/min	平均带权周转时间
J1	8:00	120	8:00	10:00	120	1	112.5	4.975
J2	8:50	50	10:00	10:50	120	2.4		
J3	9:00	10	10:50	11:00	120	12		
J4	9:50	20	11:00	11:20	90	4.5		

调度顺序为：

J1	J2	J3	J4
8:00	10:00	10:50	11:00
			11:20



2. 短作业优先（SJF）调度算法

非抢占式

思想：考虑最小化作业/进程的平均周转时间

- ❑ 算法原理：当前队列中，短作业/进程（指要求服务时间短）优先得到服务。
- ❑ 短作业优先（**short job first, SJF**）调度算法：从后备队列中选择一个或几个估计运行时间最短的作业，将它调入内存运行。
- ❑ 短进程优先（**short process first, SPF**）调度算法：从就绪队列中选择一个估计运行时间最短的作业，将处理机分配给它，使它立即执行并一直到完成，或发生某事件而被阻塞放弃处理机时，再重新调度。

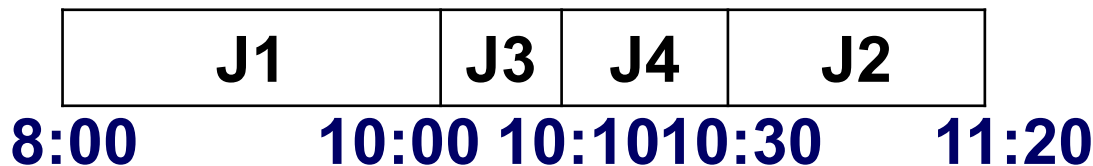


2. 短作业优先 (SJF) 调度算法

练习:

作业名	进入时间	运行时间/min	开始时间	结束时间	周转时间/min	带权周转时间	平均周转时间/min	平均带权周转时间
J1	8:00	120	8:00	10:00	120	1	95 ^ 112.5	3.25 ^ 4.975
J2	8:50	50	10:30	11:20	150	3		
J3	9:00	10	10:00	10:10	70	7		
J4	9:50	20	10:10	10:30	40	2		

调度顺序为:





2. 短作业优先（SJF）调度算法

- 优点：当多个作业同时到达时，SJF算法可使平均周转时间最短。
- 缺点：
 - 对长作业不利：长作业可能长期不被调度，会导致饥饿，甚至“饿死”。
 - 未考虑作业的紧迫性，不能保证紧迫作业（进程）会被及时调度。
 - 由于作业（进程）的长短只是根据用户所提供的估计时间而定的，致使该算法不一定能真正做到短作业优先调度。



3.2.4 优先级调度算法和高响应比优先调度算法

1. 优先级调度算法
2. 高响应比优先调度算法



1. 优先级调度算法

- **FCFS**调度算法：**作业的等待时间**就是作业的优先级，等待时间越长，其优先级越高；
- **SJF**调度算法：**作业的长短**就是作业的优先级，作业所需运行的时间越短，其优先级越高。

这两种优先级都不能反映作业的紧迫程度。

- **优先级调度算法(priority-scheduling algorithm, PSA)**:
基于**作业的紧迫程度**，由外部赋予作业相应的优先级，调度算法根据该优先级进行调度。



1. 优先级调度算法

3.3.3

□ 适用范围：

- 批处理系统的**作业调度**
- 多种操作系统的**进程调度**
- 还适用于**实时系统**

优先权作业调度：系统从**后备队列**中选择一个或几个优先权最高的作业，将它调入内存运行。

优先权进程调度：系统将处理机分配给**就绪队列**中一个优先权最高的进程。



2. 高响应比优先调度算法 非抢占式

思想：综合考虑等待时间和服务时间

□ 响应比 R_p ：实际上响应比是动态优先权

$$R_p = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$$

□ 高响应比优先调度算法(Highest Response Ratio Next, HRRN): 每次要进行作业调度时，系统首先计算后备队列中各作业的响应比，然后选择一个或若干个响应比最高的作业调入内存执行。



2. 高响应比优先调度算法

- 优点：综合了FCFS和SJF算法的优点，既考虑公平性，又考虑平均周转时间，**不会导致饥饿**。
 - 若等待时间相同，则要求服务的时间越短，其优先级越高，因此类似于SJF（短作业优先）算法；
 - 若要求服务的时间相同，作业优先权取决于其等待时间，该算法又类似于FCFS（先来先服务）算法；
 - 对于长作业的优先级，可以随着等待时间的增加而提高，当其等待时间足够长时，也可以获得处理机。
- 缺点：增加系统开销——每次调度都要计算响应比。



2. 高响应比优先调度算法

练习:

作业名	进入时间	运行时间/min	开始时间	结束时间	周转时间/min	带权周转时间	平均周转时间/min	平均带权周转时间
J1	8:00	120	8:00	10:00	120	1	102.5	3.775
J2	8:50	50	10:10	11:00	130	2.6		
J3	9:00	10	10:00	10:10	70	7		
J4	9:50	20	11:00	11:20	90	4.5		

调度顺序为:

J1	J3	J2	J4
8:00	10:00	10:10	11:00
		11:20	



3.3 进程调度

- 3.3.1 进程调度的任务、机制与方法
- 3.3.2 轮转调度算法
- 3.3.3 优先级调度算法
- 3.3.5 多级反馈队列调度算法



3.3.1 进程调度的任务、机制与方法

1. 进程调度任务

- (1) 保存处理机的现场信息
- (2) 按某种算法选取进程
- (3) 把处理器分配给进程



3.3.1 进程调度的任务、机制与方法

2. 进程调度机制

- (1) 排队器
- (2) 分派器
- (3) 上下文切换器

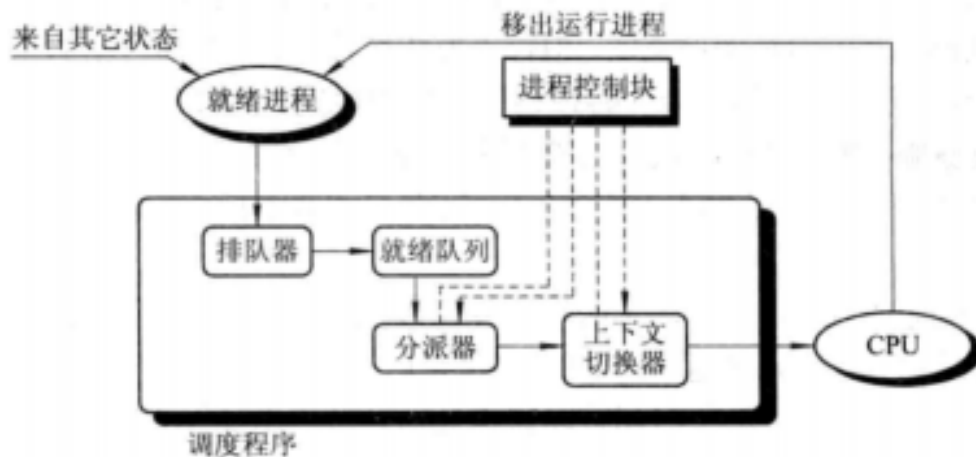


图 3-1 进程调度机制



3.3.1 进程调度的任务、机制与方法

3. 进程调度方式

(1) 非抢占方式 **只允许进程主动放弃处理机**

一旦把处理机分配给某进程后，便让它**一直执行**，直到该进程完成或发生某事件而被阻塞时，才把处理机分配给其它进程，决不允许某进程抢占已经分配出去的处理机。

优点：实现简单，系统开销小。

缺点：适用于批处理系统；难于满足紧急任务的要求，不适用分时/实时系统。



3.3.1 进程调度的任务、机制与方法

3. 进程调度方式

(2) 抢占方式

允许调度程序根据某种原则，**暂停某个正在执行的进程**，将已分配给该进程的**处理机重新分配给另一进程**。

抢占原则有：优先权原则、短作业优先原则、时间片原则。



3.3.2 轮转调度算法

在分时系统中，最简单也是较常用的是**基于时间片的轮转（round robin, RR）调度算法**。

□ **算法思想：**该方法采取非常公平的处理机分配方式，让就绪队列上的每个进程每次仅运行一个时间片。

1. 轮转法的基本原理
2. 进程切换时机
3. 时间片大小的确定



1. 轮转法的基本原理 抢占式

系统将所有就绪进程按**FCFS策略**排成一个就绪队列。系统可设置**每隔一定时间**（如30 ms）便产生一次中断（即时钟中断），去激活**进程调度程序**进行调度，**把CPU分配给队首进程**，并**令其执行一个时间片**。当它运行完毕后，又把处理机分配给就绪队列中新的队首进程，也让它执行一个时间片。

思考：

- （1）用于作业调度还是进程调度？ **进程调度**
- （2）是否会导致饥饿？ **不会**



2. 进程切换时机

在RR调度算法中，应在何时进行进程的切换，可分为两种情况：

① 若一个时间片尚未用完，正在运行的进程便已经完成，就立即激活调度程序，将它从就绪队列中删除，再调度就绪队列中队首的进程运行，并启动一个新的时间片。

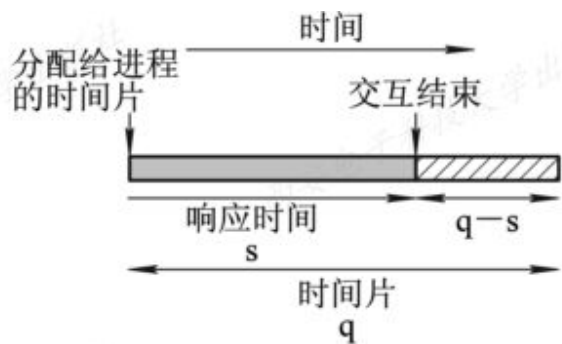
② 在一个时间片用完时，计时器中断处理程序被激活。如果进程尚未运行完毕，调度程序将把它送往就绪队列的末尾。



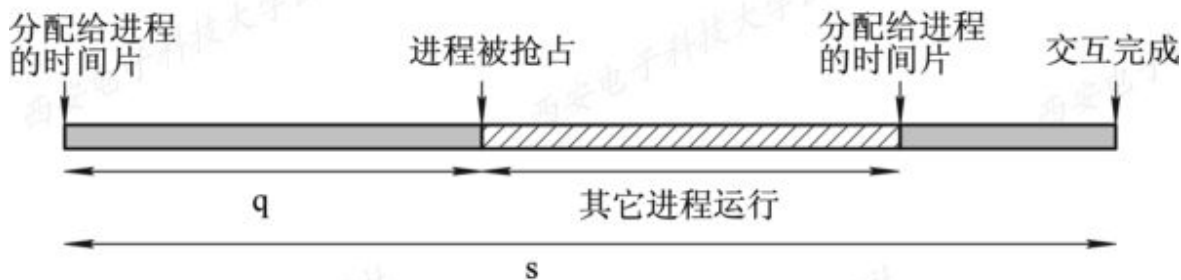
3. 时间片大小的确定

在轮转算法中，时间片的大小对系统性能有很大的影响。

图3-2给出了时间片大小对响应时间的影响，其中图（a）是时间片略大于典型交互的时间，而图（b）是时间片小于典型交互的时间。



(a) 时间片大于交互时间



(b) 时间片小于交互时间



3. 时间片大小的确定

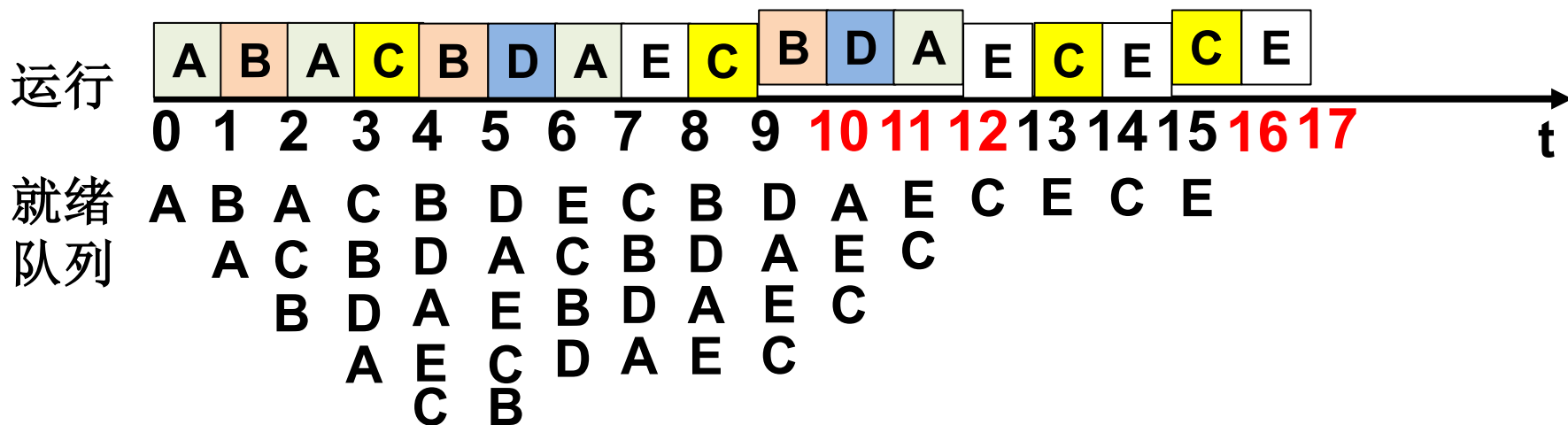
图3-3:

进程 \ 时间片	名称	A	B	C	D	E	平均
	到达时间	0	1	2	3	4	
	服务时间	4	3	4	2	4	
RR q=1	完成时间						
	周转时间						
	带权周转时间						
RR q=4	完成时间						
	周转时间						
	带权周转时间						

如果时间片太小，会导致进程切换过于频繁，系统将花费大量时间处理进程切换。

3. 时间片大小的确定

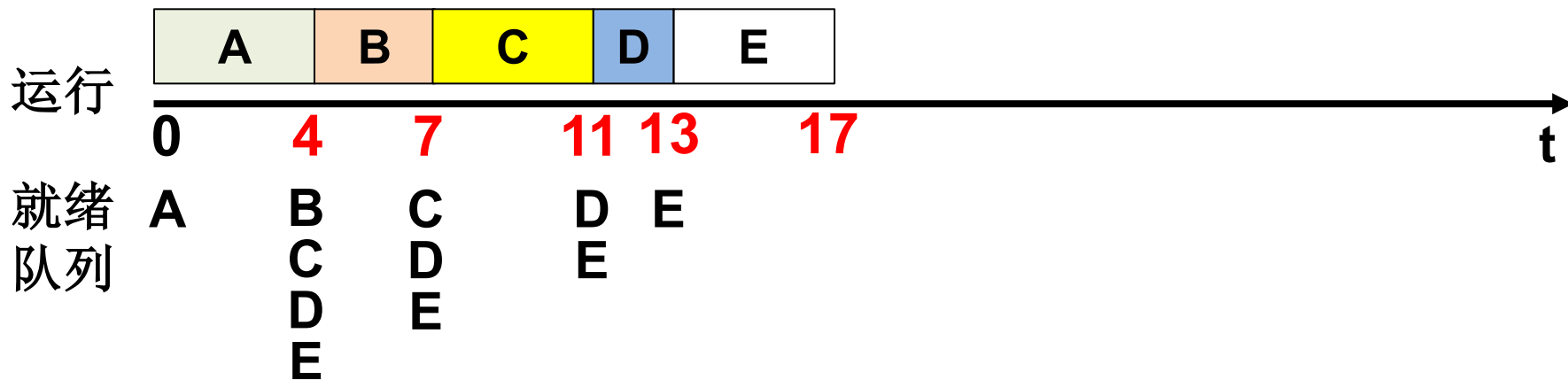
进程 时间片	名称	A	B	C	D	E	平均
	到达时间	0	1	2	3	4	
RR q=1	服务时间	4	3	4	2	4	
	完成时间	12	10	16	11	17	
	周转时间	12	9	14	8	13	11.2
	带权周转时间	3	3	3.5	4	3.25	3.35



如果时间片太大，使得每个进程都可以在一个时间片内完成。
则时间片轮转调度算法退化为先来先服务调度算法。

3. 时间片大小的确定

进程 \ 时间片	名称	A	B	C	D	E	平均
	到达时间	0	1	2	3	4	
	服务时间	4	3	4	2	4	
RR q=4	完成时间	4	7	11	13	17	
	周转时间	4	6	9	10	13	8.4
	带权周转时间	1	2	2.25	5	3.25	2.7





3.3.3 优先级调度算法

1. 优先级调度算法类型

2. 优先权的类型

思想：考虑实际情况的需要和紧迫度

优先调度优先级最高的作业/进程

优先级调度算法能更好地满足紧迫作业的要求。常用于实时系统中，以及对实时性能要求较高的批处理系统和分时系统中。



1. 优先级调度算法类型

❑ 非抢占式优先级算法 当前进程主动放弃处理机时发生调度。

一旦把处理机分配给就绪队列中优先级最高的进程后，该进程便一直执行下去，直到完成，或因发生某事件使该进程放弃处理机时，系统方可再将处理机重新分配给另一个优先级最高的进程。

❑ 抢占式优先级算法 当前进程主动放弃处理机时，就绪队列更新导致发生抢占时，都会触发调度。

把处理机分配给就绪队列中优先级最高的进程，使之执行，但在其执行期间，只要出现了另一个优先级更高的进程，系统就立即停止当前进程的执行，重新将处理机分配给新的优先级最高的进程。



2. 优先权的类型

1) 静态优先级：在创建进程时确定的，且在进程整个运行期间保持不变。一般用某一范围内的一个**整数来表示优先级**。

□ 确定优先级常用的三个依据：

- 进程类型：系统进程（如接收进程、对换进程、磁盘I/O进程等）的优先权高于一般用户进程的优先级。
- 进程对资源的需求：对资源要求少的进程赋予较高的优先级。
- 用户要求：这是由用户进程的紧迫程度和用户所付费用的多少来确定优先级的。

□ 优点：简单易行，系统开销小。

□ 缺点：优先权低的作业（进程）可能长期得不到调度。

饥饿



2. 优先权的类型

2) 动态优先级：优先级随进程的推进，或随其等待时间的增加而改变的，以便获得更好的调度性。例如：

- 在就绪队列中的进程，随其等待时间的增长，其优先级以速率 α 提高；
- 在采用抢占式优先级调度算法时，如果再规定当前执行进程以速率 β 下降，则可防止一个长作业长期垄断处理机

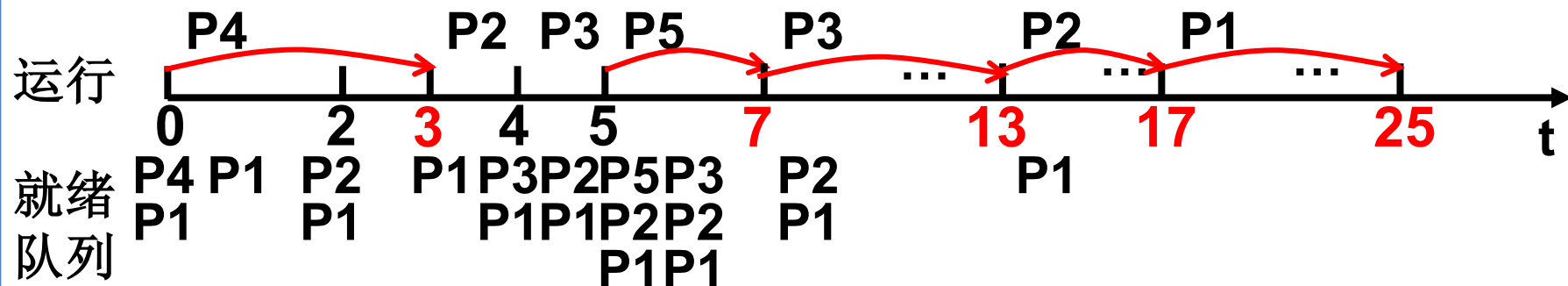
抢占式优先级调度算法：每次调度时选择当前已到达且优先级最高的进程。因此，当前进程主动放弃处理机，及就绪队列更新导致发生抢占时，都会触发调度。

例：对于某进程集合，给出了到达时间、运行时间和优先级（规定优先级数字越大，优先级越高），如下表所示。若采用静态优先级抢占式调度算法，计算各进程的运行顺序、周转时间、带权周转时间。

进程	到达时间	运行时间	优先级
P1	0	8	0
P2	2	5	1
P3	4	7	3
P4	0	3	2
P5	5	2	7



2. 优先权的类型



进程名	到达时间	运行时间	优先权	开始时间	结束时间	周转时间	带权周转时间
P1	0	8	0	17	25	25	3.125
P2	2	5	1	3	17	15	3
P3	4	7	3	4	13	9	1.29
P4	0	3	2	0	3	3	1
P5	5	2	7	5	7	2	1



3.3.5 多级反馈队列调度算法

1. 调度机制

2. 调度算法的性能

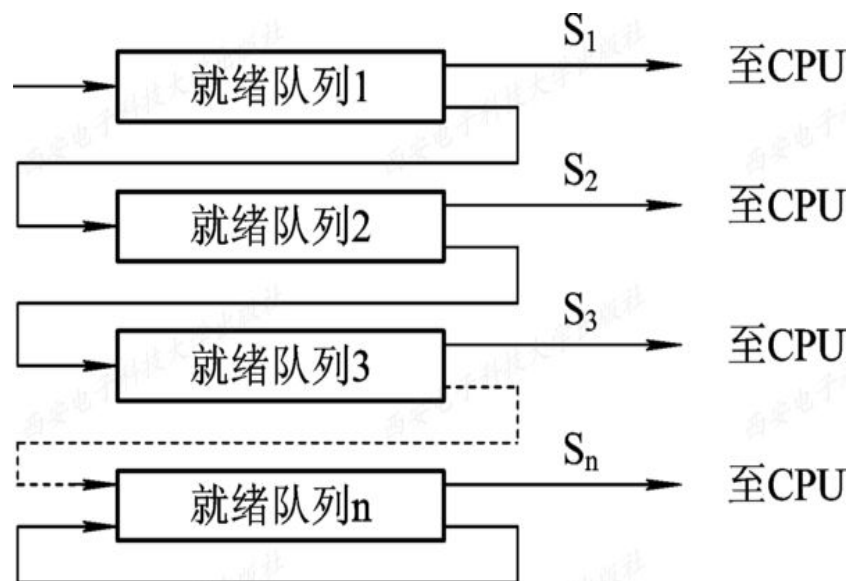
- FCFS调度算法：公平；作业/进程调度
- SJF调度算法：优先处理短作业/进程；作业/进程调度
- HRRN调度算法：均衡等待时间和运行时间；作业/进程调度
- PSA调度算法：根据紧迫程度调度；作业/进程调度
- RR调度算法：公平、轮流为进程服务；进程调度
- 多级反馈队列调度算法(multileved feedback queue, MFQ):
综合了前面多种调度算法，应用于进程调度。



1. 调度机制

(1) 设置多个就绪队列，并为每个队列赋予不同的优先级。

- 第一个队列的优先级最高，第二个次之，其余队列优先级逐个降低；
- 优先级越高的队列中，时间片越小，第 $i+1$ 个队列的时间片要比第 i 个的时间片长一倍；



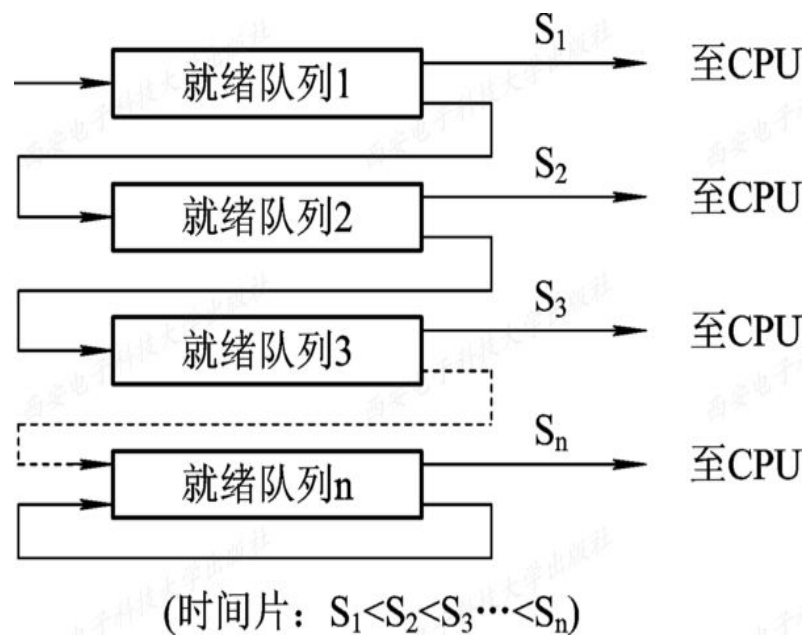
(时间片: $S_1 < S_2 < S_3 \dots < S_n$)



1. 调度机制

(2) 每个队列都采用**FCFS**算法。

- 当**新进程**进入内存后，首先将它**放入第一队列的末尾**，按FCFS原则等待调度；
- 当轮到该进程执行时，若它能在该时间片内**完成**，便可**撤离系统**。否则，即它在一个时间片结束时**尚未完成**，调度程序将其**转入第二队列的末尾**等待调度；

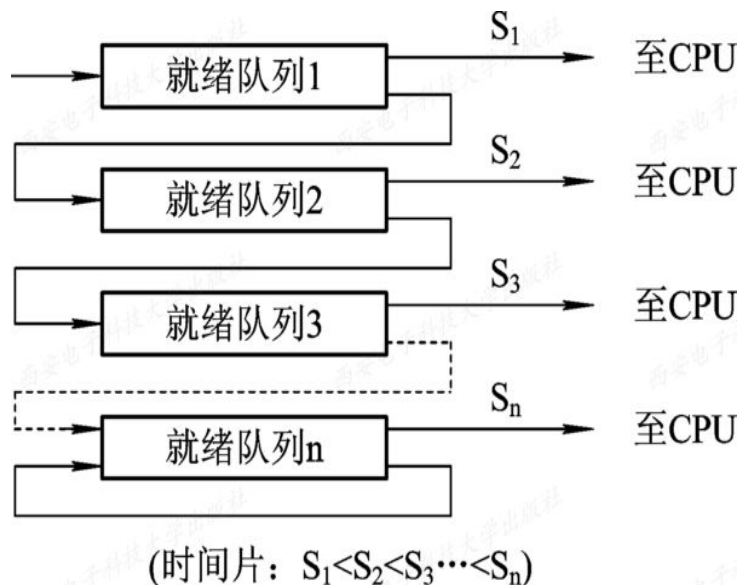




1. 调度机制

(3) 按队列优先级调度。

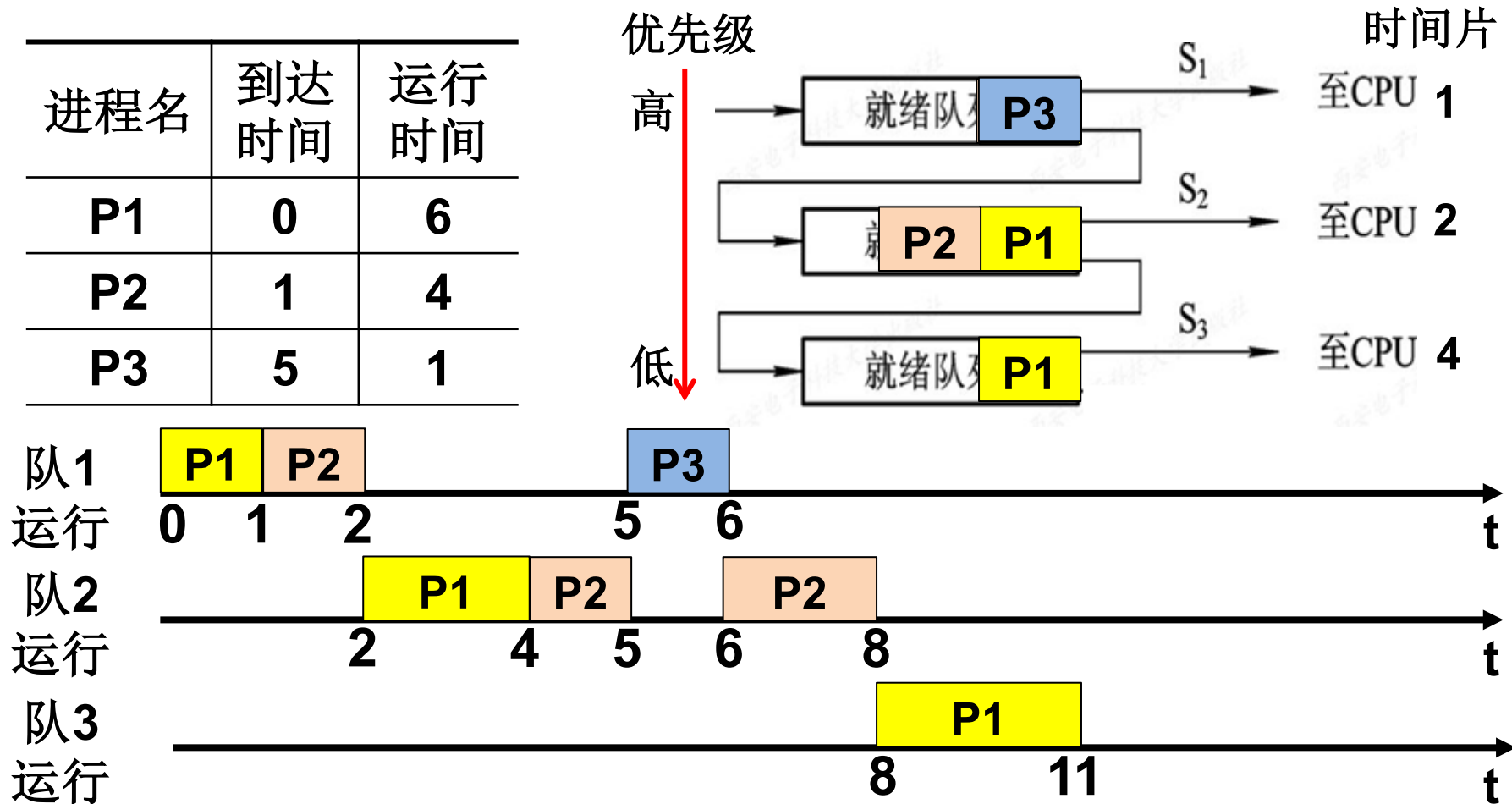
- 首先调度最高优先级队列中的进程，仅当第一队列空闲时，才调度第二队列中的进程运行……
- 如果处理机正在第 i 队列中为某进程服务时，又有新进程进入任一优先级较高的队列，此时须立即把正在运行的进程放回到第 i 队列的末尾，而把处理机分配给新到的高优先级进程。



多级反馈队列调度算法举例

例：各进程到达就绪队列的时间、需要服务时间如下表所示。
使用多级反馈队列调度算法，分析进程执行过程。

进程名	到达时间	运行时间
P1	0	6
P2	1	4
P3	5	1





2. 调度算法的性能

在多级反馈队列调度算法中，如果规定第一个队列的时间片略大于多数人机交互所需之处理时间时，便能较好地满足各种类型用户的需要。

(1) 终端型用户

提交的作业多属于交互型作业，通常较小，只需使这些作业在第一队列规定的时间片内完成，便可使终端型用户感到满意。

(2) 短批处理作业用户

对于稍长的短作业，也只需在第二和第三队列中各执行一时间片完成，其周转时间仍然较短。

(3) 长批处理作业用户

长作业，将依次在第1, 2, ..., n个队列中运行，然后再按轮转方式运行。用户不必担心其作业长期得不到处理。



小组任务：调度算法总结

调度算法	思想	原理	调度方式	适用场景	优缺点	导致饥饿
FCFS						
SJF						
HRRN						
PSA						
RR						
MFQ						



思考题

假定要在—台处理机上执行如下表所示的作业，且这些作业在时刻0以J1、J2、J3、J4、J5的顺序到达。

① 说明分别使用**FCFS**、**SJF**以及非抢占式优先调度算法时，这些作业的执行顺序。（假设数值越小，优先权高）

② 针对上述每种调度算法，求平均周转时间和平均带权周转时间。

作业	执行时间	优先级
J1	10	3
J2	1	1
J3	2	3
J4	1	4
J5	5	2



思考题

解答：作业在时刻0以J1、J2、J3、J4、J5的顺序到达。

① 使用**FCFS**调度算法时，作业的执行顺序；

② 平均周转时间和平均带权周转时间。

作业	到达时间	执行时间	FCFS算法调度顺序： J1 J2 J3 J4 J5					
			开始时间	结束时间	周转时间	带权周转时间	平均周转时间	平均带权周转时间
J1	0	10	0	10	10	1	13.4	7.26
J2	0	1	10	11	11	11		
J3	0	2	11	13	13	6.5		
J4	0	1	13	14	14	14		
J5	0	5	14	19	19	3.8		



思考题

解答：作业在时刻0以J1、J2、J3、J4、J5的顺序到达。

- ① 使用**SJF**调度算法时，作业的执行顺序；
- ② 平均周转时间和平均带权周转时间。

作业	到达时间	执行时间	SJF算法调度顺序: J2 J4 J3 J5 J1					
			开始时间	结束时间	周转时间	带权周转时间	平均周转时间	平均带权周转时间
J1	0	10	9	19	19	1.9	7	1.74
J2	0	1	0	1	1	1		
J3	0	2	2	4	4	2		
J4	0	1	1	2	2	2		
J5	0	5	4	9	9	1.8		



思考题 假设数值越小，优先权高

解答：作业在时刻0以J1、J2、J3、J4、J5的顺序到达。

① 使用非抢占式优先调度算法时，作业的执行顺序；

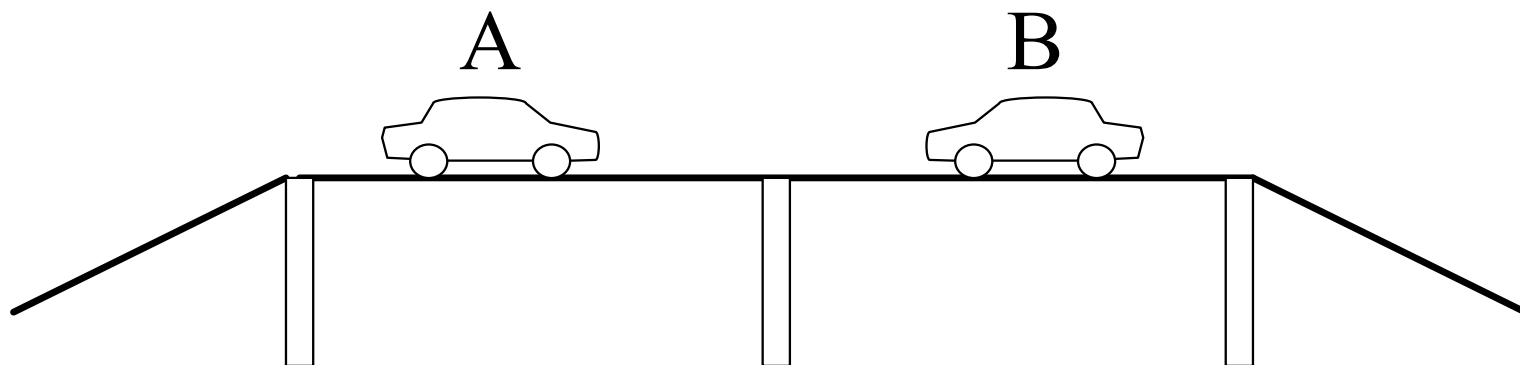
② 平均周转时间和平均带权周转时间。 **J2 J5 J1 J3 J4**

作业	到达时间	执行时间	优先级	非抢占式优先调度算法调度顺序：					
				开始时间	结束时间	周转时间	带权周转时间	平均周转时间	平均带权周转时间
J1	0	10	3	6	16	16	1.6	12	6.36
J2	0	1	1	0	1	1	1		
J3	0	2	3	16	18	18	9		
J4	0	1	4	18	19	19	19		
J5	0	5	2	1	6	6	1.2		



3.5 死锁概述

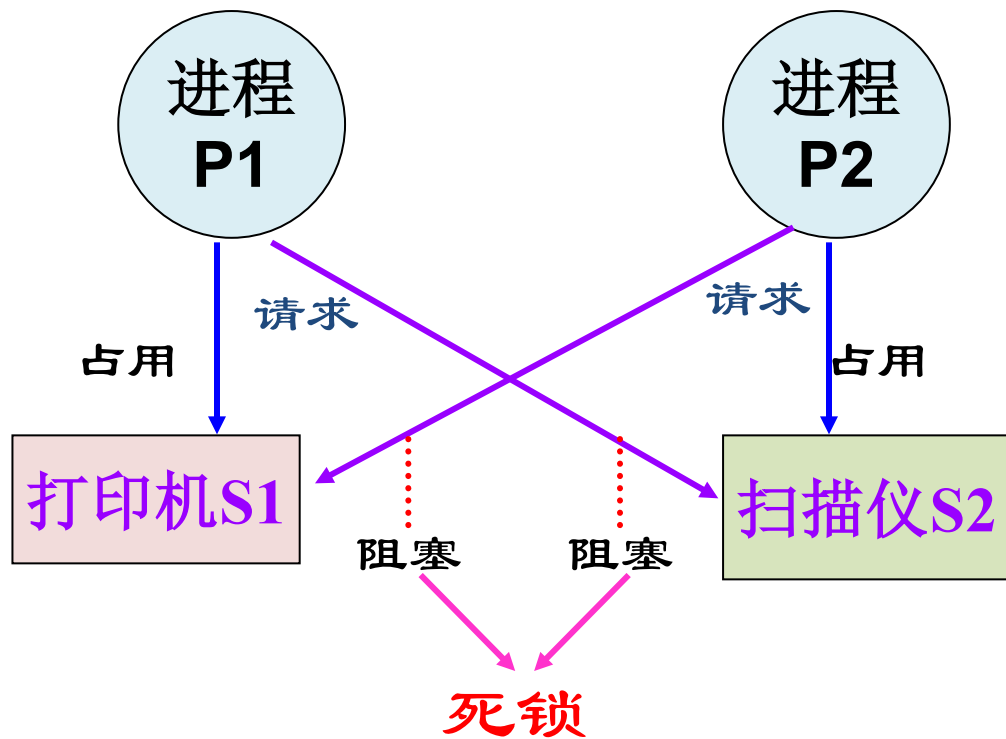
例1 两辆车过单车道桥的僵局：





3.5 死锁概述

例2 竞争外部设备。设系统中有打印机S1、扫描仪S2各一台，进程P1、P2的申请如下：





3.5 死锁概述

□死锁的定义：

多个进程在运行过程中因**争夺资源而造成的一种僵局**，当进程处于这种僵局状态时，若无外力作用，它们都将无法再向前推进。（第三版）

如果一组进程中的每一个进程都在等待仅由该组进程中其他进程才能引发的事件，那么该组进程是死锁的。（第四版）



3.5 死锁概述

□ 3.5.1 资源问题

□ 3.5.2 计算机系统死锁

□ 3.5.3 死锁的定义、必要条件和处理方法



3.5.1 资源问题

1. 可重用性资源和消耗性资源

(1) 可重用性资源

- 可供用户重复使用多次的资源；
- 对资源的请求和释放通常都是利用系统调用实现；
- 系统中大部分资源都属于可重用性资源。

(2) 可消耗性资源

- 又称为临时性资源，在进程运行期间，由进程动态地创建和消耗；
- 通常由生产者进程创建，由消费者进程消耗；
- 如消息、中断信号、同步信号等。



3.5.1 资源问题

2. 可抢占性资源和不可抢占性资源

(1) 可抢占性资源（可剥夺性资源）

- 指某进程获得这类资源后，该资源可被其他进程或系统抢占；
- 如：CPU和内存。

(2) 不可抢占性资源（非剥夺性资源）

- 系统一旦把这类资源分配给某进程后，不能强行收回，只能进程用完后自行释放；
- 如：光盘、打印机等。



3.5.2 计算机系统中的死锁

1. 竞争不可抢占性资源引起死锁 资源不够，进程抢夺资源，产生死锁。
2. 竞争可消耗性资源引起死锁
3. 进程推进顺序不当引起死锁

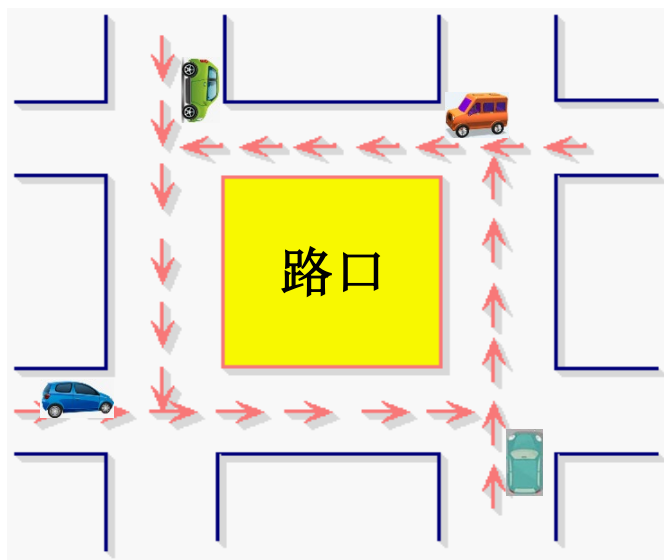
进程运行过程中，请求资源和释放资源的顺序不当，产生死锁。



3.5.2 计算机系统中的死锁

1. 竞争不可抢占性资源引起死锁

例1. 日常生活中常有许多有关死锁的事件。



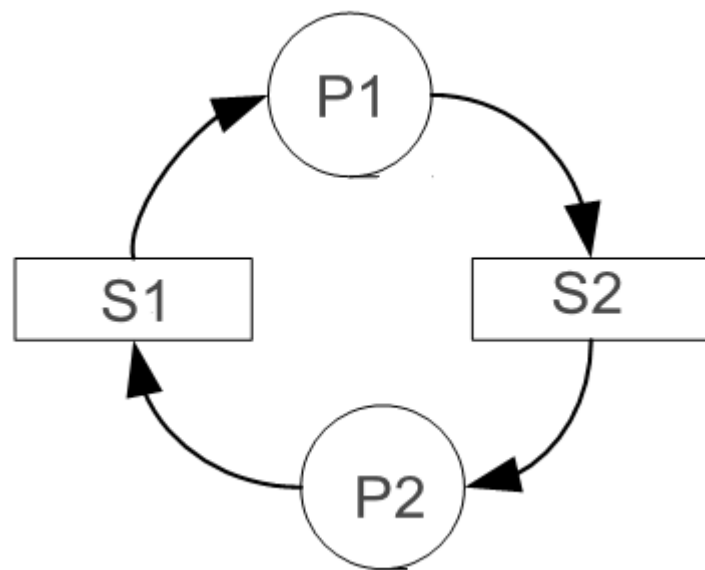
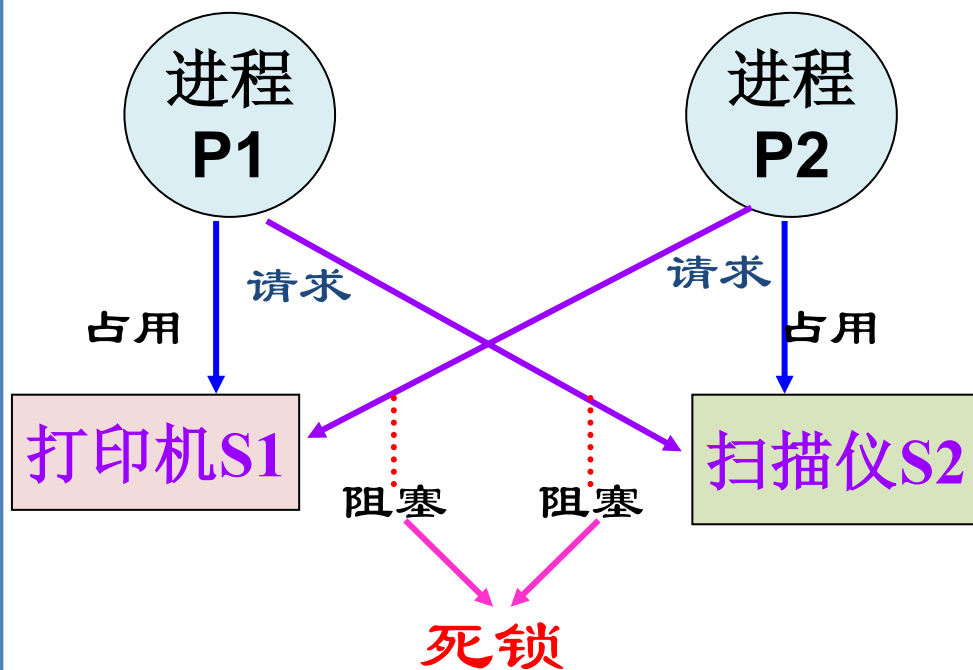
假设都不改变行车方向，各路车队等待的事件都不会发生。若不采用特殊方法，它们将永远停留在这“井”字形的路上，而处于**死锁**状态。



3.5.2 计算机系统中的死锁

1. 竞争不可抢占性资源引起死锁

例2. 竞争外部设备。设系统中有打印机S1、扫描仪S2各一台，进程P1、P2的申请如下：





3.5.2 计算机系统中的死锁

2. 竞争可消耗性资源引起死锁

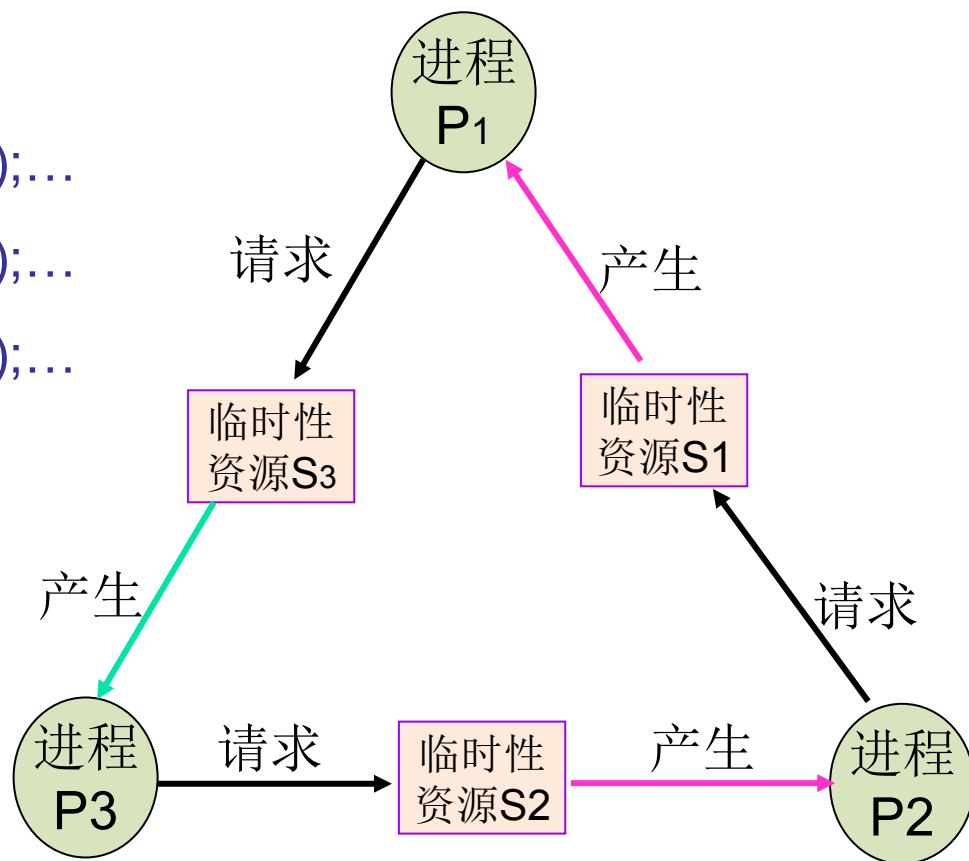
若按下列顺序进行：

P_1 : ... Release(S_1); Request(S_3);...

P_2 : ... Release(S_2); Request(S_1);...

P_3 : ... Release(S_3); Request(S_2);...

则不会产生“死锁”。





3.5.2 计算机系统中的死锁

2. 竞争可消耗性资源引起死锁

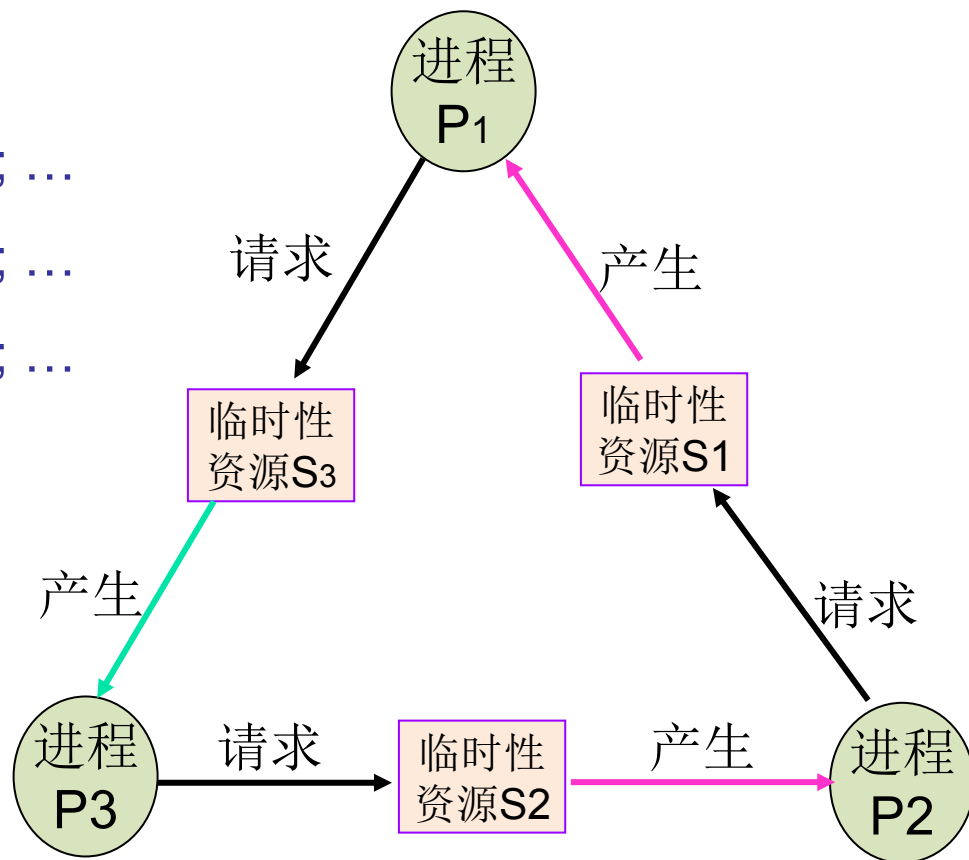
若按下列顺序进行：

P_1 : ...Request(S_3); Release(S_1); ...

P_2 : ...Request(S_1); Release(S_2); ...

P_3 : ...Request(S_2); Release(S_3); ...

则很可能发生“死锁”





3.5.2 计算机系统中的死锁

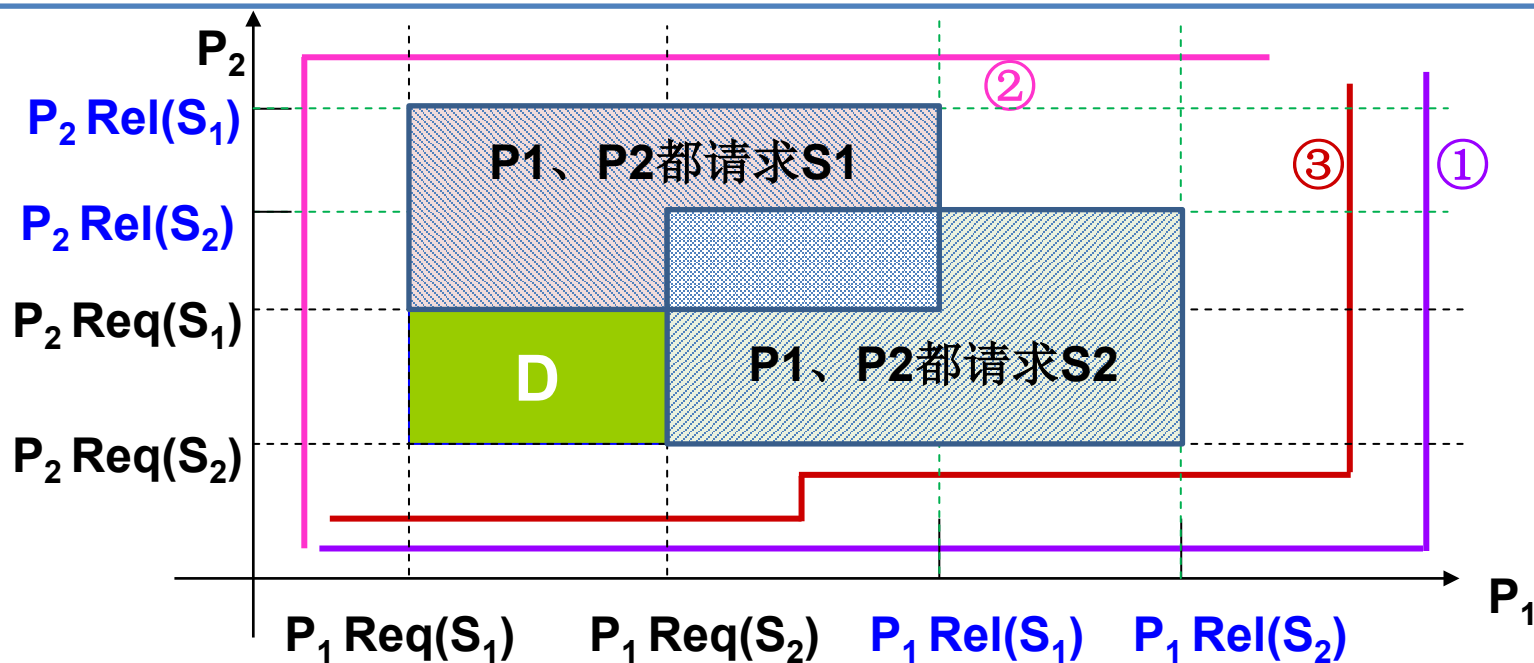
3. 进程推进顺序不当引起死锁

假设进程 P_1 、 P_2 共享一台打印机 S_1 和一台扫描仪 S_2 。由于进程具有异步特征，这就使得进程 P_1 、 P_2 可能按下列两种顺序向前推进：

- (1) 进程推进顺序合法
- (2) 进程推进顺序非法



(1) 进程推进顺序合法



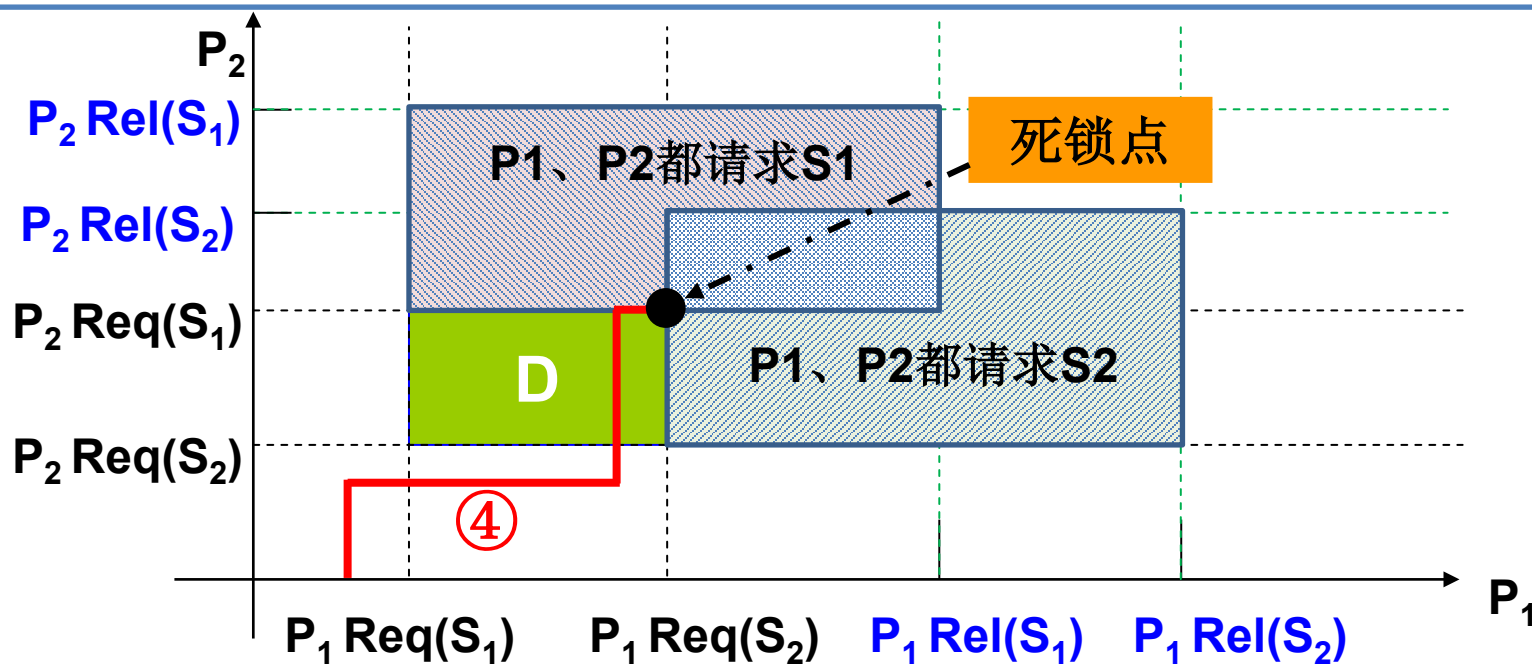
曲线①: $P_1:\text{Request}(S_1) \rightarrow P_1:\text{Request}(S_2) \rightarrow P_1:\text{Release}(S_1) \rightarrow P_1:\text{Release}(S_2) \rightarrow P_2:\text{Request}(S_1) \rightarrow P_2:\text{Request}(S_2) \rightarrow P_2:\text{Release}(S_1) \rightarrow P_2:\text{Release}(S_2)$ ，两个进程可顺利完成，不会产生“死锁”。

同样②、③中两个进程也不会产生“死锁”。

进入不安全区后，继续推进，迟早会进入死锁状态。



(2) 进程推进顺序非法



按曲线④： $P_1:Request(S_1) \rightarrow P_2:Request(S_2) \rightarrow P_1:Request(S_2)$ 阻塞
 $\rightarrow P_2:Request(S_1)$ 阻塞，则两个进程间互相产生了阻塞，从而产生“死锁”。

把区域D称为是“不安全区”，此时 P_1 占有 S_1 ， P_2 占有 S_2 。



3.5.3 死锁的定义、必要条件和处理方法

1. 死锁的定义

多个进程在运行过程中因**争夺资源而造成的一种僵局**，当进程处于这种僵局状态时，若无外力作用，它们都将无法再向前推进。（第三版）

如果一组进程中的每一个进程都在等待仅由该组进程中其他进程才能引发的事件，那么该组进程是死锁的。（第四版）



3.5.3 死锁的定义、必要条件和处理方法

2. 产生死锁的必要条件

补：充分条件和必要条件：

充分条件： 如果A发生必然导致B发生，则A为B的充分条件。

必要条件： 如果B发生必然导致A发生；且如果A不发生必然导致B不发生，则A为B的必要条件。



3.5.3 死锁的定义、必要条件和处理方法

2. 产生死锁的必要条件

(1) 互斥条件

对分配到的资源进行排他性使用。

(2) 请求和保持条件

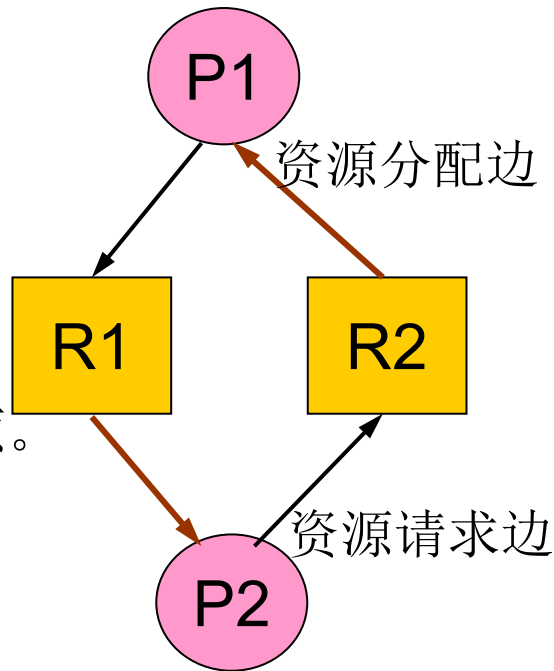
保持占有已获得资源，请求被其他进程占有的资源。

(3) 不可抢占条件

已获得资源在未使用完之前不能被抢占。

(4) 循环等待条件

发生死锁时，必然存在“进程-资源”循环链。



四个必要条件中只要有一个条件不满足，都不会形成“死锁”。



3.5.3 死锁的定义、必要条件和处理方法

3. 处理死锁的基本方法

(1) 预防死锁

- 方法：设置某些限制条件，去破坏产生死锁的四个必要条件中的一个或几个条件，来预防发生死锁。
- 缺点：可能导致系统资源利用率和系统吞吐量的降低。——较严格的限制条件

(2) 避免死锁

- 方法：在资源动态分配过程中，用某种方法防止系统进入不安全状态，从而避免死锁。
- 优点：目前在较完善的系统中，常用此方法来避免死锁。——只要较弱的限制条件



3.5.3 死锁的定义、必要条件和处理方法

3. 处理死锁的基本方法

(3) 检测死锁

- **方法：**允许系统在运行过程中发生死锁，但可通过系统设置的检测机构，及时检测出死锁的发生，然后采取适当的措施，从系统中将已发生的死锁清除掉。

(4) 解除死锁

- **方法：**与检测死锁相配套的措施。常用的方法是撤消或挂起一些进程，以便回收一些资源，分配给已处于阻塞状态的进程，使之转为就绪状态，以继续运行。——实现上难度最大。



3.6 预防死锁

□ 3.6.1 破坏“请求和保持”条件

□ 3.6.2 破坏“不可抢占”条件

□ 3.6.3 破坏“循环等待”条件

实现采取某种限制措施，破坏产生死锁的必要条件

由于资源自身特性，不能破坏“互斥”共享条件



3.6.1 破坏“请求和保持”条件

□ 保证：当一个进程在请求资源时，不能持有不可抢占资源。

1. 第一种协议：预分配资源策略

- 规定：在一个进程开始执行之前，必须**一次性申请**其整个运行过程中所需的**全部资源**。
- 优点：简单、易行、安全。
- 缺点：资源浪费严重、利用率低；进程经常会发生饥饿现象。

2. 第二种协议：空手申请资源策略

- 规定：每个进程**仅在不占有资源时才可以申请资源**。一个进程可能需要申请并使用某些资源，在它们申请另外附加资源之前，必须先释放当前分到的全部资源。
- 优点：第一种协议的改进



3.6.2 破坏“不可抢占”条件

- ❑ **规定：** 当一个已持有某些不可抢占资源的进程，提出新的资源请求而不能得到满足时，该进程必须释放已经占有的所有资源，待以后需要时再重新申请。
- ❑ **缺点：** 复杂，延长进程周转时间，增加系统开销，减低系统吞吐量，代价大。



3.6.3 破坏“循环等待”条件

- 方法：对系统所有资源类型进行线性排序，并赋予不同的号，采用**有序资源分配策略**，即，所有进程对资源的请求应严格按资源序号递增顺序提出。

例如：1磁带机，2扫描仪，3打印机，4绘图仪，...，10

P1:
申请1
申请3
...

P2:
申请2
申请4
...

P3 P10

一个进程只有已占有小编号资源时，才有资格申请更大编号的资源。即，已持有大编号资源的进程不能逆向申请小编号资源，从而不会发生循环等待现象。



3.6.3 破坏“循环等待”条件

□ 优点：资源利用率高，系统吞吐量。

□ 缺点：

- 为系统中各种资源类型分配序号时，必须相对稳定，从而限制了新设备的增加；
- 会出现作业使用的资源顺序与系统规定的顺序不同的情况，造成资源的浪费。



3.7 避免死锁

□ 3.7.1 系统安全状态

□ 3.7.2 利用银行家算法避免死锁

资源动态分配过程中，防止系统进入不安全状态，
以避免发生死锁。



3.7.1 系统安全状态

1. 安全状态

系统能按某种进程顺序 $(P_1, P_2, \dots, P_n)$ 为每个进程分配其所需资源，直至满足每个进程对资源的最大需求，使每个进程都可**顺利地**完成。此时称 $(P_1, P_2, \dots, P_n)$ 为安全序列。

2. 不安全状态

如果系统无法找到一个安全序列，则系统处于不安全状态。

- 并非所有不安全状态都处于死锁状态；但发生死锁时一定处于不安全状态；
- 当系统进入不安全状态后，便可能（或者早晚会）进入死锁状态；
- 只要系统处于安全状态，便可避免进入死锁状态。



安全状态例子

设系统中有3个进程P1、P2、P3，共有12台磁带机。进程需求资源以及T0时刻已分配情况如下表所示。

进程	最大需求	T0时刻已分配	还需要	可用数
P1	10	5	5	3
P2	4	2	2	
P3	9	2	7	

经分析，T0时刻系统是安全的，因为这时存在一个安全序列(P2, P1, P3)，即只要系统按此进程序列分配资源，就能使每一个进程都顺利完成。

- 只要系统处于安全状态，便可避免进入死锁状态。

- 并非所有不安全状态都处于死锁状态；但发生死锁时一定处于不安全状态；
- 当系统进入不安全状态后，便可能（或者早晚会）进入死锁状态；

设系统中有3个进程P1、P2、P3，共有12台磁带机。进程需求资源以及T0、T1时刻分配情况如下表所示。

T0时刻安全序列(P2, P1, P3)

未按照安全序列分配资源

进程	最大需求	T0时刻已分配	T1时刻已分配	还需要	可用数
P1	10	5	5	5	2
P2	4	2	2	2	
P3	9	2	3	6	

T1时刻无法再找到一个安全序列，此时系统处于不安全状态。

如果不按照安全顺序分配资源，则系统可能由安全状态进入不安全状态。



3.7.2 利用银行家算法避免死锁

- 最有代表性的避免死锁算法是Dijkstra E.W 于1968年提出的**银行家算法**。该算法可用于预测一笔贷款业务是否会引起银行资金周转问题，是否是安全的。
- 银行的资金类似于计算机系统的资源，贷款业务类似于计算机的资源分配。**该算法能预测一次资源分配对计算机系统是否是安全的。**
- 银行家算法的核心思想：资源动态分配前先判断此次分配是否会导致系统进入不安全状态，若否，则同意分配；否则拒绝分配。这样可防止系统进入不安全状态，从而避免死锁。



3.7.2 利用银行家算法避免死锁

□ 银行家算法模型：假定小城镇的一个银行家拥有资金，数量为 Σ ，被 N 个客户共享。银行家对客户提出下列约束条件：

- 每个客户必须预先说明自己所**要求的最大资金量**；
- 每个客户每次提出**部分资金量申请**和获得分配；
- 如果银行满足了某客户对资金的最大需求量，那么，客户在资金运作**结束后**，应在**有限时间内全部归还银行**。



3.7.2 利用银行家算法避免死锁

□ 算法实现过程:

- 每个新客户（进程）进入系统时，必须申明自己所需要每种类型资源的最大数量，其数量不可超过系统所拥有的资源总量；

客户对资源最大需求数量 **系统拥有资源总量**

- 某个时刻 T_i ，客户（进程）请求一组资源时，系统必须首先确定是否有足够分配的资源；

T_i 时刻客户对资源需求数量 **系统可利用资源数量**

系统可利用资源数量 = 系统拥有资源总量 - T_i 时刻客户已分配资源数量

- 若有，进一步计算将资源分配给客户（进程）后，是否会进入不安全状态。如果不会，才分配资源；否则，让进程等待。



数据结构举例

设系统中有3个进程P1、P2、P3，共有3类资源：7个R1、2个R2、5个R3。进程最大需求资源以及T1时刻已分配情况如下表所示。系统拥有(R1, R2, R3)总量: (7, 2, 5)

资源 进程	最大需求			T1时已分配			T1时需求量			剩余可用量		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	2	2	2			
P2	6	1	3	6	1	2	0	0	1	0	1	1
P3	4	2	2	0	0	2	4	2	0			

分析可知：T1时刻，剩余可用资源够P2使用。且P2使用完毕后释放所有资源，使剩余可用资源变为(7, 2, 4)，足以保证P1、P3顺利推进。所以存在安全序列<P2, P1, P3>

进程 i 对第 j 类资源需求数量为：

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

1. 银行家算法中的数据结构

(1) 可利用资源向量**Available**

一个含有m个元素的数组，其中每个元素代表一类可用资源数目，m是资源种类数。其初始值为系统中资源最大可用数。

(2) 最大需求矩阵**Max**

一个 $n \times m$ 矩阵，用于表示系统中n个进程中的每个进程对m类资源的最大需求数。

(3) 分配矩阵**Allocation**

一个 $n \times m$ 矩阵，用于表示系统中每类资源当前已分配给每个进程的资源数。

(4) 需求矩阵**Need**

一个 $n \times m$ 矩阵，用于表示每个进程尚需的各类资源数。



2. 银行家算法步骤

设 $Request_i$ 是进程 P_i 的请求向量， $Request_i[j] = K$ 表示进程 P_i 需要 K 个 R_j 类的资源。当 P_i 发出资源请求后，系统按下述步骤进行检查：

(1) 判断 $Request_i[j]$ 与 $Need[i,j]$ 的关系

- 若 $Request_i[j] > Need[i,j]$ ，出错。因为它需要的资源数已超过它所声明的最大值；
- 若 $Request_i[j] \leq Need[i,j]$ ，转向步骤 (2)。

(2) 判断 $Request_i[j]$ 与 $Available[j]$ 的关系

- 若 $Request_i[j] > Available[j]$ ，表示无足够资源， P_i 须等待；
- 若 $Request_i[j] \leq Available[j]$ ，转向步骤 (3)。



2. 银行家算法步骤

(3) 系统**试探**着把资源分配给进程 P_i ，并修改下面的数值：

$$\text{Available}[j] = \text{Available}[j] - \text{Request}_i[j]$$

$$\text{Allocation}[i,j] = \text{Allocation}[i,j] + \text{Request}_i[j]$$

$$\text{Need}[i,j] = \text{Need}[i,j] - \text{Request}_i[j]$$

(4) 系统执行**安全性算法**，检查此次资源分配后，系统是否处于安全状态。若安全，才真正将资源分配给进程 P_i ，以完成本次分配；否则，将本次资源分配作废，恢复原来的资源分配状态，让进程 P_i 等待（阻塞）。



3. 安全性算法

判断安全性的算法可描述如下：

(1) 设置两个向量

- ① 工作向量Work：表示系统目前可提供的各类资源数，有m个元素。在执行本算法开始时， $Work = Available$ ；
- ② 标志向量Finish：表示系统是否有足够资源分配给进程，使之运行完成。开始时 $Finish[i]=false$ ($i=1,2,\dots,n$)，表示进程 P_i 未结束；当有足够资源分配给进程 P_i 时，再令 $Finish[i]=True$ ，表示进程 P_i 已完成。



3. 安全性算法

(2) 从进程集合中寻找一个能满足下述条件的进程:

- ① $\text{Finish}[i] = \text{False};$
- ② $\text{Need}[i, j] \leq \text{Work}[j] \ (j=1, 2, \dots, m);$

若找到，转步骤 (3) 执行；否则，执行步骤 (4)。

(3) 进程 P_i 可获得所需全部资源，可正常执行直至结束，然后释放分配给它的资源。故应执行：

$\text{Work}[j] = \text{Work}[j] + \text{Allocation}[i, j];$

$\text{Finish}[i] = \text{True};$

返回步骤 (2)

(4) 如果所有进程的 $\text{Finish}[i] == \text{True}$ 都满足，则表示系统处于安全状态；否则，系统处于不安全状态



4. 银行家算法例子

例1 系统中有5个进程 $\{P_0, P_1, P_2, P_3, P_4\}$ 和3类资源 $\{A, B, C\}$ ，各资源的数量为 $\{10, 5, 7\}$ ， T_0 时刻资源分配情况如下表所示。依次求：（1） T_0 时刻的安全性；（2） P_1 : $\text{Request}_1(1, 0, 2)$ ；（3） P_4 : $\text{Request}_4(3, 3, 0)$ ；（4） P_0 : $\text{Request}_0(0, 2, 0)$

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	3	3	2
P_1	3	2	2	2	0	0	1	2	2			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

解：（1） T_0 时刻的安全性； 安全性算法

T_0 时刻存在安全序列(P_1, P_3, P_4, P_2, P_0)，故此时系统是安全的。

资源 进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P_1	3	3	2	1	2	2	2	0	0	5	3	2	false→true
P_3	5	3	2	0	1	1	2	1	1	7	4	3	false→true
P_4	7	4	3	4	3	1	0	0	2	7	4	5	false→true
P_2	7	4	5	6	0	0	3	0	2	10	4	7	false→true
P_0	10	4	7	7	4	3	0	1	0	10	5	7	false→true

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	3	3	2
P_1	3	2	2	2	0	0	1	2	2			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

解：（2） P_1 请求资源：Request₁(1, 0, 2) 银行家算法

① Request₁(1, 0, 2) ≤ Need₁ (1, 2, 2)

② Request₁(1, 0, 2) ≤ Available(3, 3, 2)

③ 试分配资源后，修改数据结构：

Available[j] = Available[j] – Request₁ [j]

Allocation[1,j] = Allocation[1,j] + Request₁ [j]

Need[1,j] = Need[1,j] – Request₁ [j]

④ 对试分配后T₁时刻的状态进行安全性检查：

资源 进程		Max			Allocation			Need			Available		
		A	B	C	A	B	C	A	B	C	A	B	C
	P ₀	7	5	3	0	1	0	7	4	3			
T ₀ 时刻	P ₁	3	2	2	3, 0, 2			0, 2, 0					
	P ₂	9	0	2	3	0	2	6	0	0	2, 3, 0		
T ₁ 时刻	P ₃	2	2	2	2	1	1	0	1	1			
	P ₄	4	3	3	0	0	2	4	3	1			

解：④ 对试分配后 T_1 时刻的状态进行安全性检查： **安全性算法**

T_0 时刻存在安全序列 $(P_1, P_3, P_4, P_2, P_0)$ ，故此时系统是安全的。

资源 进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P_1	2	3	0	0	2	0	3	0	2	5	3	2	false→true
P_3	5	3	2	0	1	1	2	1	1	7	4	3	false→true
P_4	7	4	3	4	3	1	0	0	2	7	4	5	false→true
P_2	7	4	5	6	0	0	3	0	2	10	4	7	false→true
P_0	10	4	7	7	4	3	0	1	0	10	5	7	false→true

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	2	3	0
P_1	3	2	2	3	0	2	0	2	0			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

解： (3) P_4 : $\text{Request}_4(3, 3, 0)$

银行家算法

T_1 时刻

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3			
P_1	3	2	2	3	0	2	0	2	0			
P_2	9	0	2	3	0	2	6	0	0	2	3	0
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

① $\text{Request}_4(3, 3, 0) \leq \text{Need}_4(4, 3, 1)$

② $\text{Request}_4(3, 3, 0) \leq \text{Available}(2, 3, 0)$ 不成立，故让 P_4 等待。

解：（4） P_0 请求资源：Request₀(0, 2, 0) 银行家算法

- ① Request₀(0, 2, 0) ≤ Need₀ (7, 4, 3)
- ② Request₀(0, 2, 0) ≤ Available(2, 3, 0)
- ③ 试分配资源后，修改数据结构：

Available[j] = Available[j] - Request₁ [j]
Allocation[1,j] = Allocation[1,j] + Request₁ [j]
Need[1,j] = Need[1,j] - Request₁[j]

- ④ 对试分配后T₂时刻的状态进行安全性检查：

Available(2,1,0)已不能满足任何进程的需要，故系统进入不安全状态，所以不能为P₀分配资源，而应恢复原来的状态，让P₀等待。

T₁时刻



T₂时刻

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	7	5	3	0, 3, 0			7, 2, 3			2, 1, 0		
P ₁	3	2	2	3	0	2	0	2	0			
P ₂	9	0	2	3	0	2	6	0	0			
P ₃	2	2	2	2	1	1	0	1	1			
P ₄	4	3	3	0	0	2	4	3	1			

解：④ 对试分配后 T_2 时刻的状态进行安全性检查：安全性算法

Available(2,1,0)已不能满足任何进程的需要，故系统进入不安全状态，所以不能为 P_0 分配资源，而应恢复原来的状态，让 P_0 等待。

T_2 时刻

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	2	3	0
P_1	3	2	2	3	0	2	0	2	0			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

练习题： 电子科技大学研究生入学考试试题

假设有两类资源A和B，A类资源10个，B类资源14个，当前系统的资源分配情况如下表所示。根据分配表，回答下面两个问题：

①请填写系统的需求矩阵。

②使用银行家的算法，确定系统是否死锁状态？如果不死锁给出安全序列，如果死锁给出死锁的四个条件。

进程	Allocation		Max		Need		Available	
	A	B	A	B	A	B	A	B
P ₀	2	0	2	4	0	4	2	7
P ₁	3	2	10	2	7	0		
P ₂	1	4	5	4	4	0		
P ₃	2	1	3	1	1	0		
P ₄	0	0	4	2	4	2		



● 练习题

用银行家算法考虑下列系统状态：

资源 进程	最大需求矩阵				分配矩阵				资源总数矩阵			
A	4	1	1	1	3	0	1	1	6	3	4	2
B	0	2	1	2	0	1	0	0				
C	4	2	1	0	1	1	1	0				
D	1	1	1	1	1	1	0	1				
E	2	1	1	0	0	0	0	0				

问：

(1)此时系统是否安全？为什么？

(2)若进程B请求(0,0,1,0)，可否立即分配？

(3)此后进程E也请求(0,0,1,0)，可否分配给它？



河南大學
HENAN UNIVERSITY

明德，新民
止于至善



The End ...

